진화적 설계

Daum Corp.
백명석

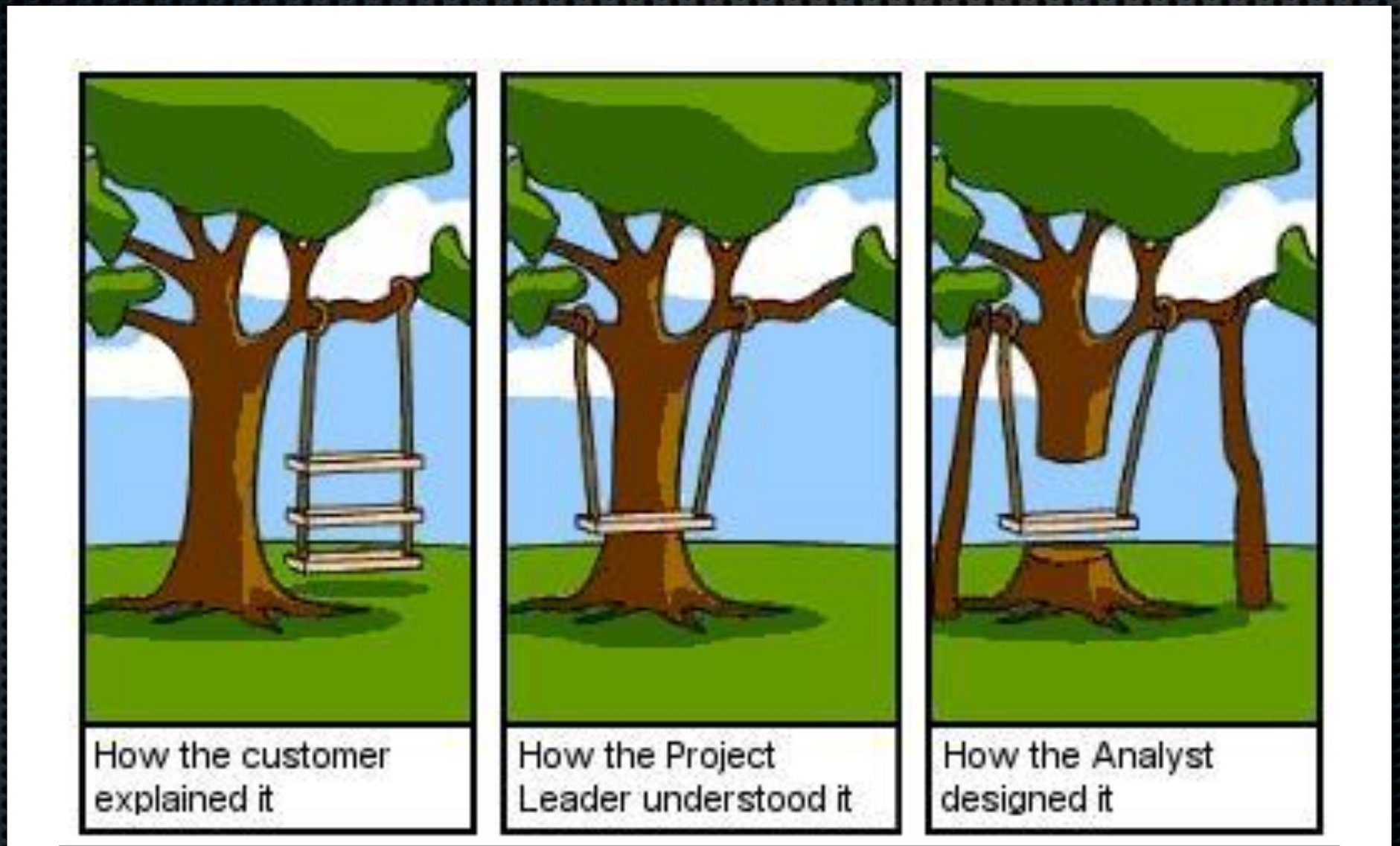
목차

- 진화적 설계란
- 과장된 SW Application
- 문제의 근원
  - 잘못된 메타포 차용
  - SW 설계란
  - 공학적 메타포로 오염된 개발 포스세스
- 진화적 설계
  - 인간의 본성 / 모토 / 현재를 위한 설계 / 피드백을 통한 진행
- 사전 설계보다 더 효과적인 방법
  - TDD / Refactoring
  - 시간 변화에 따른 생산성 변화
  - TDD & Refactoring의 효과
- SW Engineer의 나아갈 길

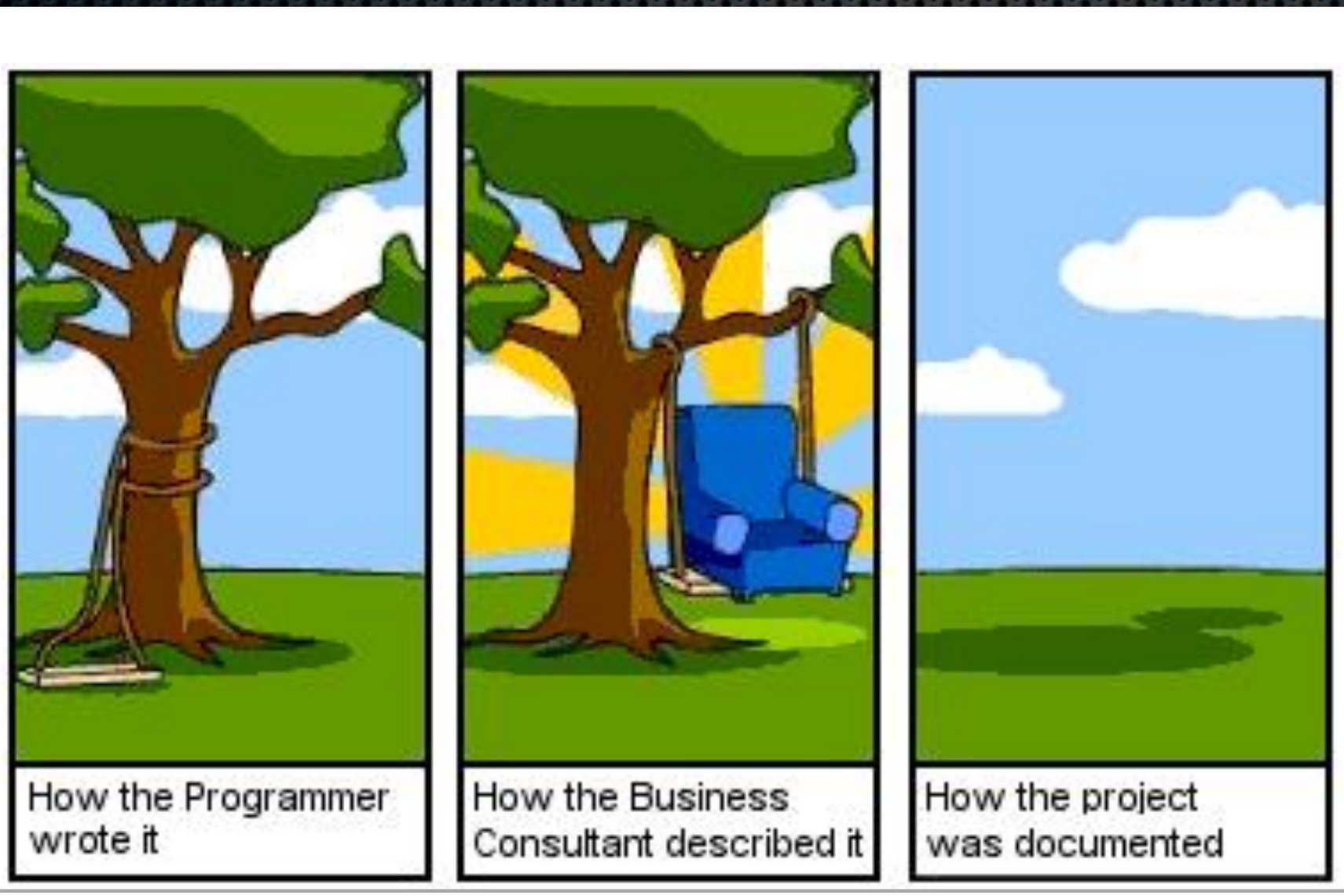
진화적 설계란

- 계획된 설계/사전 설계
  - Planned Design, Upfront Design
  - 분석 -> 설계 -> 구현 -> 테스트 -> 운영
- 진화적 설계(Evolutionary Design)
  - 필요한 만큼만
  - 분석 -> ((구현/테스트) -> 설계) -> 운영
- known unknowns / unknown unknowns

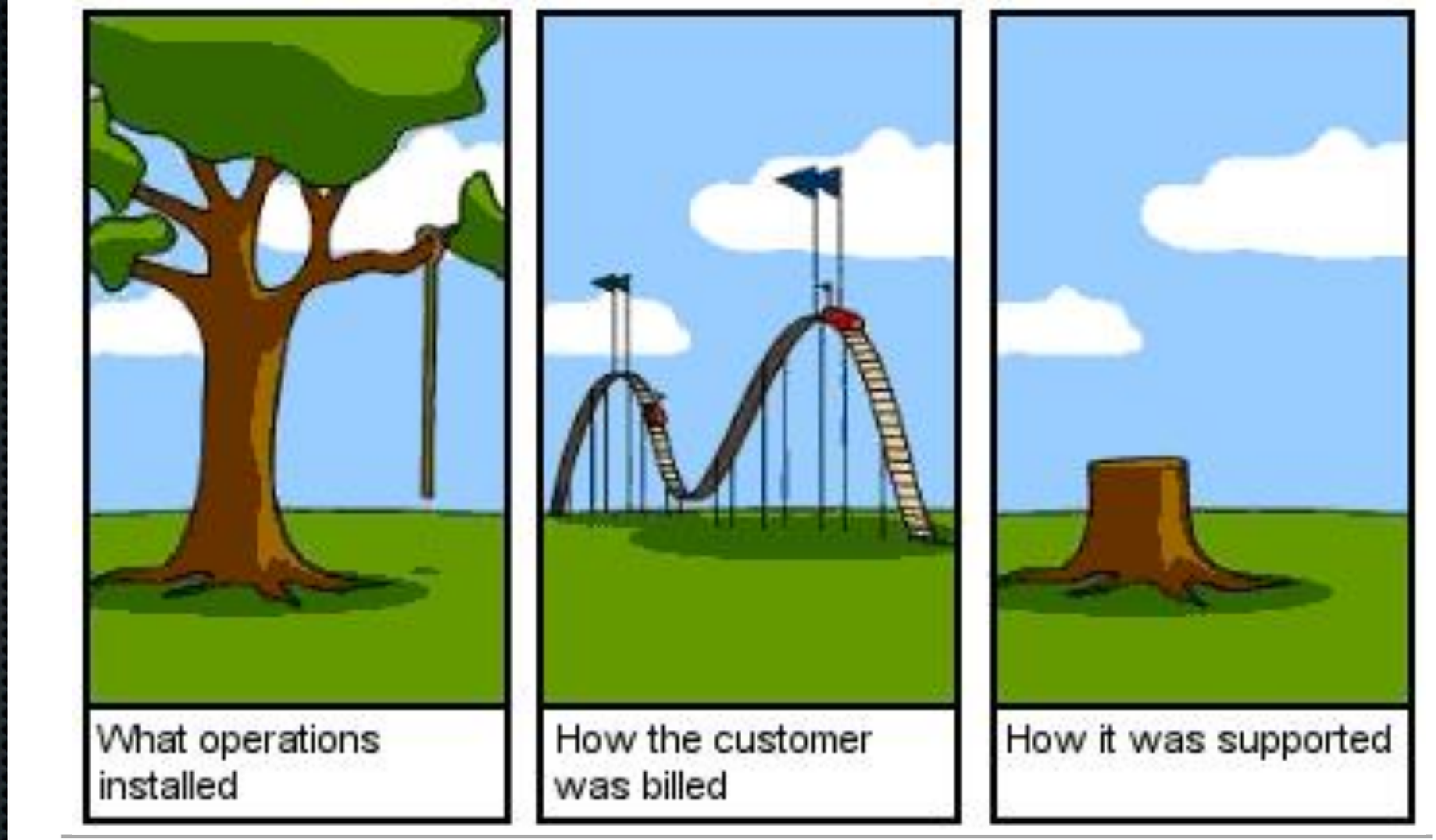
왜 진화적 설계 !! - 과장된 SW Application



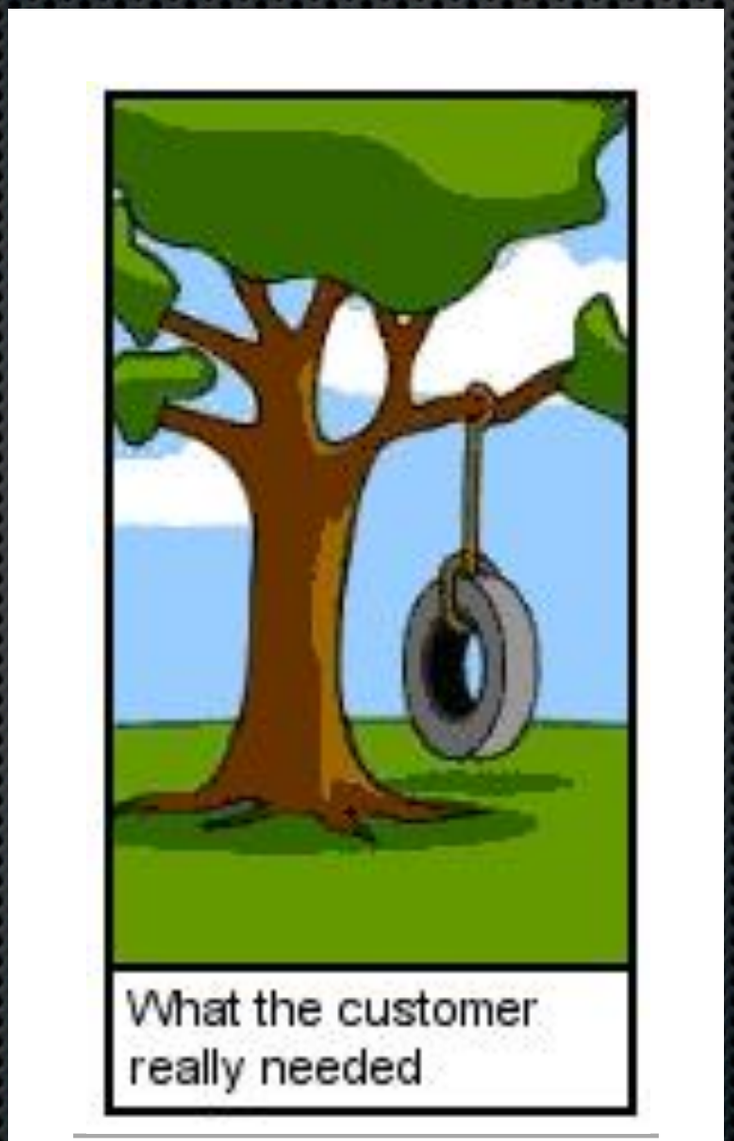
왜 진화적 설계 !! - 과장된 SW Application



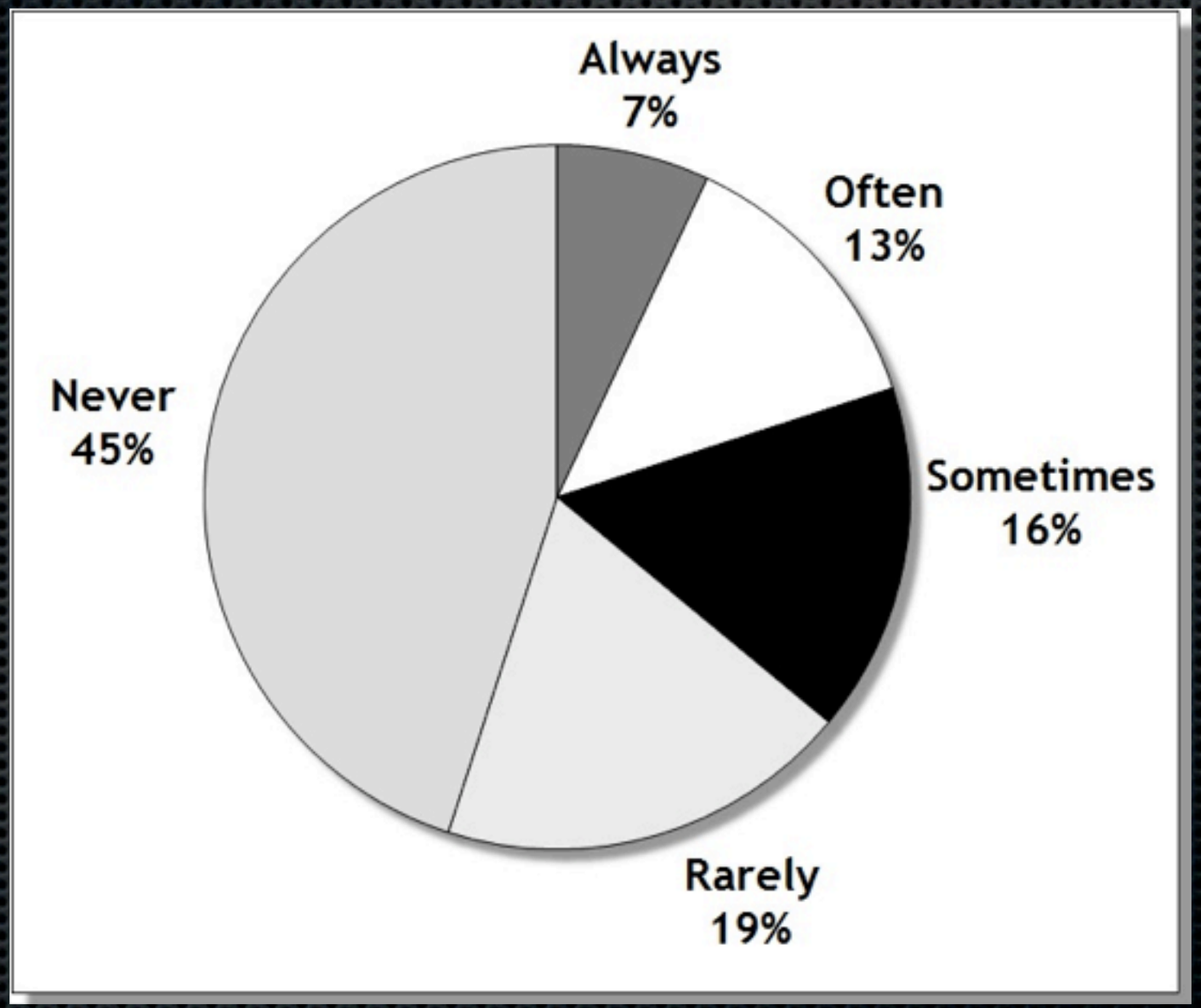
왜 진화적 설계 !! - 과장된 SW Application



왜 진화적 설계 !! - 과장된 SW Application



구현된 기능 사용율



문제의 근원

- 항상 변하는 요구사항
- 요구사항 정리 완료 -> 소프트웨어 설계
- 개발 싸이클 vs 요구사항 수정 비용
- 기간 준수 vs 요구사항 확정
  - 이후 변경 불가 고수

문제의 근원

- 요구사항을 만족시키지 못하는 이유
  - 사용해보기 전에는 원하는 기능이 뭔지 모름
  - 실제 SW를 사용해 볼 수 있는 후반에만 피드백 가능
  - 막대한 비용 수반

결과

- 생명주기 후반에 변경이 있을 경우 기간 준수 불가
- 가치가 있더라도 후반의 변경 요청 수용 안 함
- 그 결과 7%만 사용.
- 품질 저하

잘못된 메타포 차용

- 다른 공학 분야(건축)에서 메타포 차용
  - 건축가가 설계도면 완성
  - 하도급자에게 전달하여 건물을 짓도록 함
- SW 개발 프로세스에 다양한 측면에서 악영향
- 요구사항 변경은 악
  - 100층짜리 건물. 50층. 요구사항 변경 ???
- 건축: 설계 -> 구축
- SW: Coding -> Build
  - UML은 설계(coding)을 위한 전초전
- 건축은 구축에 많은 비용. SW는 Coding에 많은 비용
- 건축은 구축시 변경 불가. SW는 저렴한 구축 비용

13년 8월 30일 금

요구사항 변경은 악

설계 완료 후 건축 시작된 후 설계 변경은 많은 비용 발생 혹은 불가능

잘못된 메타포 차용

- 설계와 구현을 완전히 분리
  - 설계 완료 후 -> 코드 구현
- 구현 전 모든 설계를 확정하는 방식
  - Big Design Up-Front(BDUF)
  - 구현 과정에서 얻게되는 피드백을 통한 설계 개선 여지 원천적 차단

SW 공학과의 차이점(목적)

- 건축학
  - 설계 도면에 어그러짐 없이 건물을 건설하는 것
- 제조업
  - 설계에 부합하는 제품을 제공하는 것
- SW
  - 구체적인 피드백과 배움을 통해 설계를 성장시켜 나가는 분야
  - 프로젝트에 합류한지 일주일만에 설계한 모듈
  - vs.
  - 6개월 후의 재 설계한 모듈의 품질 차이

SW 설계

- 시간의 흐름 속에서 얻어진 지식을 토양으로
- 최적의 설계를 배양하는 배움의 과정
- 현재의 요구사항을 만족하는 지에 대한 가장 확실한 피드백을 받을 수 있는 유일한 방법은 해당 설계를 구현해 보는 것

SW

- 2 Values of SW
  - it's behavior(현재의 SW가 현재의 요구사항 만족)
  - 지속적으로 변화는 요구사항을 수용하는 것
- 2 Kinds of Code
  - 기계가 이해하는 코드
  - 사람이 이해하는 코드
    - 잘 쓰여진 산문처럼 읽혀지는 코드

공학적 메타포로 오염된 개발 프로세스

- 구현을 통한 피드백 원천 차단
- 과도한 사전 설계에 순응하는 코드 생성 요구
- 어떠한 장치에도 불구하고 변경되는 요구사항
- 요구사항 변경에 대한 설계자의 최후 해결책
  - 변경 가능한 모든 항목들을 미리 설계
  - 불필요한 유연성/복잡성
  - 머피의 법칙

변경을 수용하라

- 요구사항 변경은 오류가 아니다
  - 사용자가 SW에 대해 더 잘 알게되었다는 환영의 신호
  - 개발할 SW에 대해 더 잘 알게되었다는 긍정의 신호
  - 변경을 제한하면 7%의 비참한 결과
- 개발 과정을 둔하게 하는 요인
  - 충분한 정보가 없는 상태에서 불안정한 결정 내림
  - 내린 결정을 변경하지 않으려고 함

변경을 수용하라

- 사전 설계 방식
  - 명확하지 않음에도 요구사항 베이스 라인 확정
  - 요구사항이 변경될 수 있는 상황에서 미리 설계를 고정
    - 불필요할 수도 있는 유연성 추가
    - 코드 복잡도 높임
  - 요구 사항 변경이 악이라는 가정에서 출발한 결과

진화적 설계 / 인간의 본성

- 명확한 오늘에 충실하기 보다 불확실한 내일에 대비
- 제어할 수 없는 것을 제어할 수 있다고 착각
- 요구사항 변경, 과도한 사전 설계 발생

진화적 설계 / 모토

- 내일이 아니라 오늘에 충실하자
  - make it pass vs refactoring
  - pair
- 오늘 주어진 정보가 충실하지 않다면 내일로 미뤄라
- 오늘은 오늘 주어진 정보만으로 가능한 작업을 하라

진화적 설계 / 현재를 위한 설계

- 오늘 구현에 필요한 요구사항에 대해서만 고객과 커뮤니케이션 하라
- 내일 구현할 요구사항은 간단한 목록 정도로만 책상 한편에 치워두라
- 언제 변경될 지, 언제 구현될 지 모르는 요구사항에 너무 많은 시간을 소비하지 마라
- 오늘의 요구사항을 만족시킬 정도만 설계하라
- 오늘 필요로 하지 않는 유연성은 추가하지 말라
  - 아예 필요 없을 수도 있다.
  - 불필요한 유연성은 복잡도를 증가시킴

진화적 설계 / 피드백 통한 진행

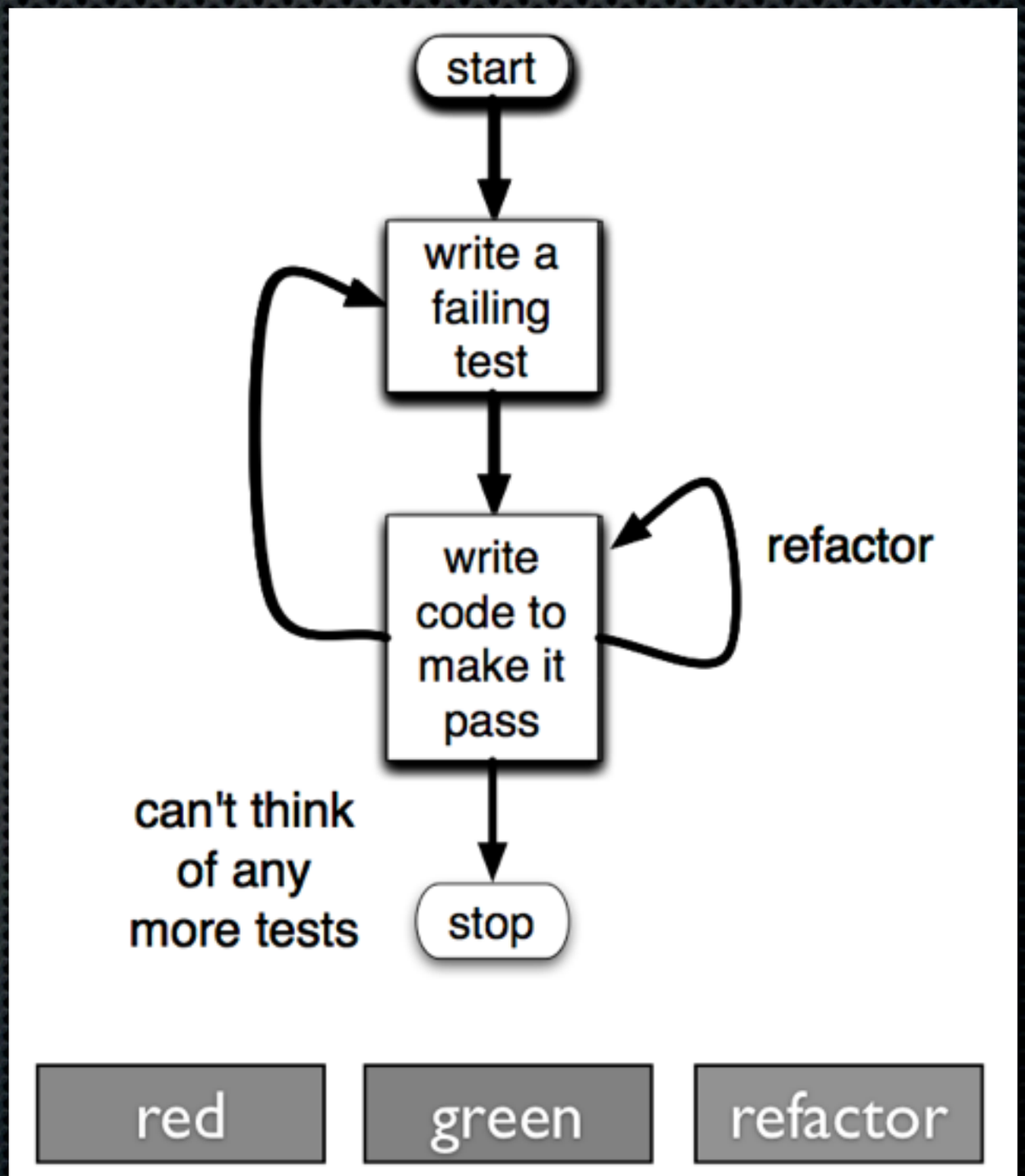
- 피드백 주기가 짧을 수록 오늘을 위해 작업하고 있는 지 여부를 더 확실하게 알 수 있다.
- 고객의 피드백 주기를 줄이는 방법
  - 더 짧은 주기로 SW를 배포
- 설계에 대한 피드백 주기 줄이기
  - 짧은 개발 주기(Iteration)로 나누고
  - 각 iteration에서 구현해야 하는 부분에 대해서만 설계
  - 곧장 구현에 돌입
- 점진적(Incremental) 개발로 요구사항 수용

사전 설계보다 더 효과적인 방법

- 설계 후 구현하는 대신 구현 후 설계
  - TDD(Test Driven Development)
    - Test Driven Design
    - Test First Development
  - Refactoring

프랙티스와 원칙 - TDD

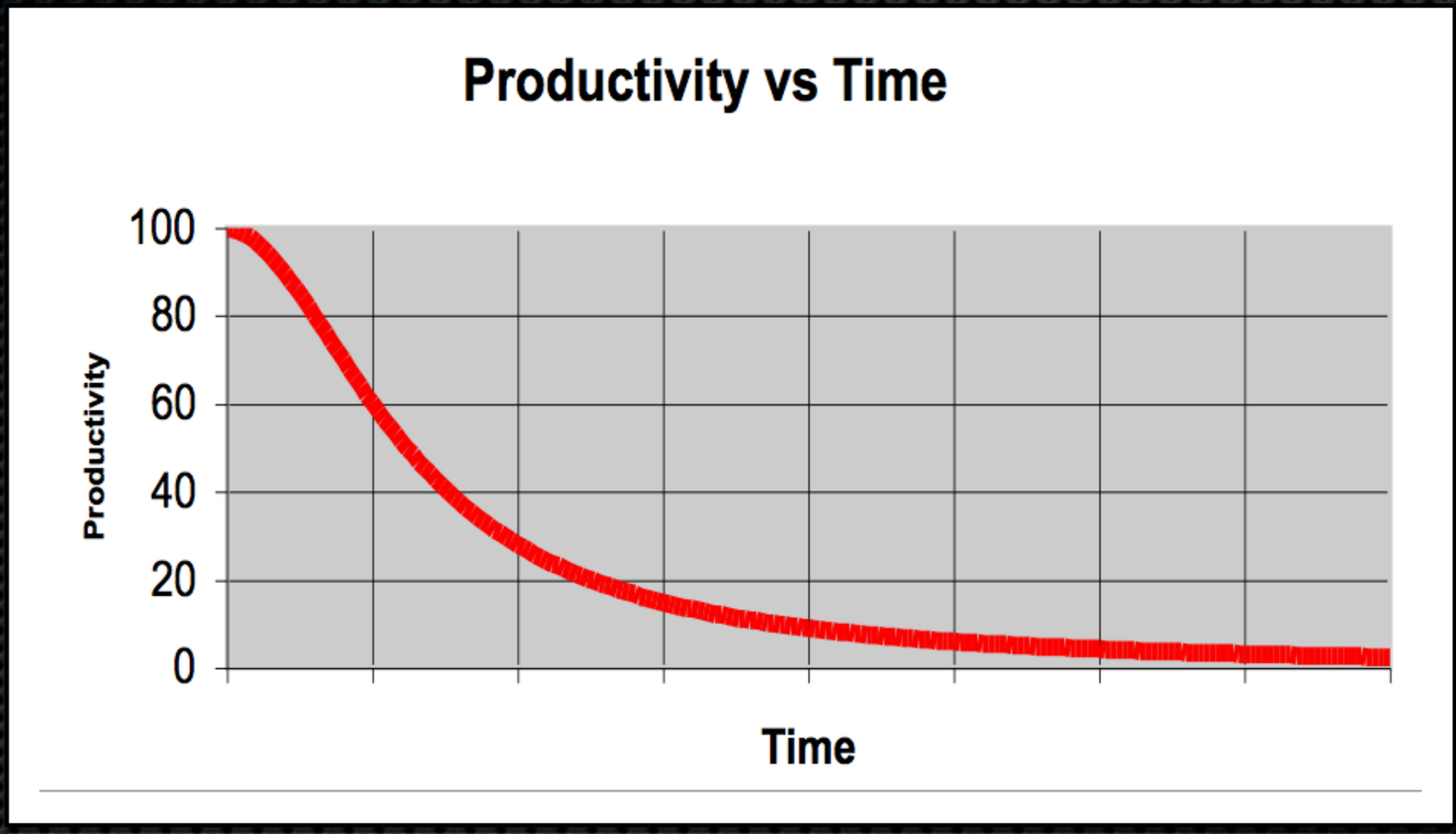
- 절차
  - 테스트 작성
  - 테스트 통과시키기
    - (중복 허용)
  - 리팩토링
- 결과
  - Clean code that works



프랙티스와 원칙 -Refactoring

- 화려하지만 과도한 설계 vs 투박하지만 적합한 설계
- 적합한 설계에 이를 수 있는 방법
- 안전망(TDD)
- CleanCode
- Boy scout Rule

시간의 변화에 따른 생산성 변환



2가지 지침을 따른다면

- 과도한 설계와 분석 마비 상태에서 허비하는 시간을 줄이고
- 다양한 기능을 포함하는 품질 높은 코드를 빠른 시간 안에 작성할 수 있게 됨

나아갈 길

- 2가지 프랙티스를 준수한다고 진화적 설계의 길에 들어서는 것은 아님
  - 설계를 진화시키기 위해서는
    - 훌륭한 설계 원칙과 원리, 다양한 이론에 대한 학습
    - 이를 지속적으로 적용시킬 수 있는 기술이 필요
  - 이론적 토대 위에 서지 못한 프랙티스
    - 엉성하고 겉만 화려한 구호에 지나지 않음